

Python Lab Assesment

Name: Pranav Anand

Reg No: 23MIC0006

Problem #1: Design a Python class called Student to track a student's academic performance.

Requirements:

Constructor (`_init_`)

Accepts two parameters: name (string) and student_id (string or int).

Initializes an empty list scores to store the student's marks.

Instance Variables:

name → student's name

student_id → student's unique ID

scores → list of numeric scores (initially empty)

Problem #2: Design a Python class called Gene to represent a DNA sequence.

The class should:

Store a gene's ID and sequence.

Provide methods to:

Print the gene's ID and sequence.

Count the occurrence of a specific base (A, T, G, C).

Calculate the GC content (fraction of G and C bases).

Get and set the sequence, ensuring that RNA bases (U) are not allowed.

Answers start in next pg proolly

```
In [1]: #23MIC0006
        #Pranav Anand
```

```
In [72]: #Problem 1
class Student:
    scores=[]
    def __init__(self, name, student_id):
        self.name=name
        self.student_id=student_id
        self.scores=[]

    def add_score(self, score):
        self.scores.append(score)

    def calculate_average(self):
        if len(self.scores)>0:
            a=0
            for i in self.scores:
                a+=i

            return (a/len(self.scores))

        else:
            return 0

    def get_final_grade(self):
        a=self.calculate_average()
        if a>=60:
            return "Pass"
        else:

            return "Fail"
```

```
In [45]: student1 = Student("Alice", 101)
print(student1.name)      # Expected: Alice
print(student1.student_id) # Expected: 101
print(student1.scores)    # Expected: []
```

```
Alice
101
[]
```

```
In [46]: student1.add_score(75)
student1.add_score(85)
student1.add_score(55)
print(student1.scores)    # Expected: [75, 85, 55]
```

```
[75, 85, 55]
```

```
In [47]: print(student1.calculate_average())
        # Expected: (75 + 85 + 55) / 3 = 71.67
```

```
71.66666666666667
```

```
In [48]: print(student1.get_final_grade())
        # Expected: Pass (average = 71.67 ≥ 60)
```

```
Pass
```

```
In [49]: student2 = Student("Bob", 102)
student2.add_score(40)
student2.add_score(50)
print(student2.calculate_average()) # Expected: 45.0
print(student2.get_final_grade())   # Expected: Fail
```

45.0

Fail

```
In [50]: student3 = Student("Charlie", 103)
print(student3.calculate_average()) # Expected: 0
print(student3.get_final_grade())   # Expected: Fail
```

0

Fail

```
In [62]: #Problem 2
```

```
class Gene:

    def __init__(self, idd, sequence):
        self.id=idd
        self.sequence=sequence

    def print_id(self):
        print(f"Wow! My id is: {self.id}")

    def print_sequence(self):
        print(f"Wow! My sequence is: {self.sequence}")

    def base_composition(self, thing):
        a=0
        for i in self.sequence:
            if i==thing:
                a+=1
        return a

    def gc_content(self):
        g=0
        c=0
        for i in self.sequence:
            if i=="G":
                g+=1
            elif i=="C":
                c+=1

        return ((g+c)/len(self.sequence))

    def set_seq(self, seq):
        for i in seq:
            if i=="U":
                return "Sorry, no RNA allowed"

        self.sequence=seq

    def get_seq(self):
        return self.sequence
```

```
In [63]: gene1 = Gene(101, "ATGCGT")
# Expected output (from constructor prints):
# I'm a new Gene object!
# My constructor got a param: 101
# Assign that param to my id instance variable...
# Similarly, assigning to my sequence instance variable...
```

```
In [64]: gene1.print_id()
# Expected: My id is: 101
gene1.print_sequence()
# Expected: My sequence is: ATGCGT
```

Wow! My id is: 101

Wow! My sequence is: ATGCGT

```
In [66]: print(gene1.base_composition("A"))
# Expected: 1
print(gene1.base_composition("G"))
# Expected: 2
print(gene1.base_composition("T"))
# Expected: 2
```

1
2
2

```
In [67]: print(gene1.gc_content())
# Sequence: ATGCGT → G=2, C=1 → GC count=3
# Length=6 → Expected: 3/6 = 0.5
```

0.5

```
In [68]: print(gene1.get_seq())
# Expected: ATGCGT
```

ATGCGT

```
In [69]: gene1.set_seq("ATATGC")
print(gene1.get_seq())
# Expected: ATATGC
```

ATATGC

```
In [70]: gene1.set_seq("AUGC")
# Expected: AssertionError → "Sorry, no RNA allowed."
```

Out[70]: 'Sorry, no RNA allowed'

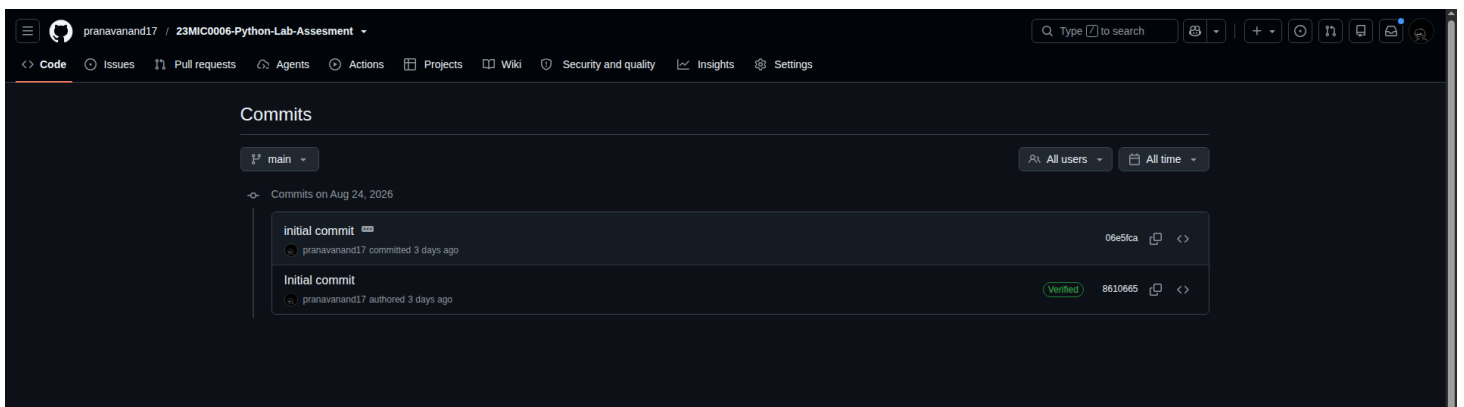
In []:

Advanced Python Programming Lab Task 2

7 Tasks from Multithreading Practice

Github link:

<https://github.com/pranavanand17/23MIC0006-Python-Lab-Assesment>



```
In [1]: #23MIC0006
        #Pranav Anand
```

```
In [2]: # Task 1
        # Creating a simple thread using threading module

import threading
import time

start = time.perf_counter()

# Function executed by the thread
def work():
    print("Sleeping for 1 second")
    time.sleep(1)
    print("Finished sleeping")

# Create and start the thread
thread = threading.Thread(target=work)
thread.start()

# Wait for the thread to finish
thread.join()

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

```
Sleeping for 1 second
Finished sleeping
Finished in 1.03 second(s)
```

```
In [3]: # Task 2
        # Creating multiple threads using threading module

import threading
import time

start = time.perf_counter()

# Function executed by each thread
def work():
    print("Sleeping for 1 second")
    time.sleep(1)
    print("Finished sleeping")

threads = []

# Create and start 10 threads
for count in range(10):
    thread = threading.Thread(target=work)
    thread.start()
    threads.append(thread)

# Wait for all threads to complete
for thread in threads:
    thread.join()
```

```
end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

```
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished in 1.13 second(s)
```

```
In [4]: # Task 3
        # Creating multiple threads and passing arguments

import threading
import time

start = time.perf_counter()

# Function that accepts the sleeping time
def work(seconds):
    print(f"Sleeping for {seconds} second")
    time.sleep(seconds)
    print("Finished sleeping")

threads = []

# Create 50 threads
for count in range(50):
    thread = threading.Thread(target=work, args=(1,))
    thread.start()
    threads.append(thread)

# Wait for all threads
for thread in threads:
    thread.join()

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

localhost:8888/doc/tree/23MIC0006 MULTITHREADING.ipynb

[illegible]

```
In [5]: # Task 4  
# Creating threads using concurrent futures module  
  
import concurrent.futures  
import time  
  
start = time.perf_counter()  
  
# Function executed by the threads  
def work(seconds):  
    print(f"Sleeping for {seconds} seconds")  
    time.sleep(seconds)  
    return f"Finished sleeping for {seconds} seconds"  
  
# Create a thread pool  
with concurrent.futures.ThreadPoolExecutor() as pool:  
    # Submit two tasks
```

```

    first = pool.submit(work, 1.5)
    second = pool.submit(work, 1.5)

    # Get the results
    print(first.result())
    print(second.result())

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

Sleeping for 1.5 seconds
 Sleeping for 1.5 seconds
 Finished sleeping for 1.5 seconds
 Finished sleeping for 1.5 seconds
 Finished in 1.52 second(s)

```

In [6]: # Task 5
        # Creating multiple threads using list comprehension

import concurrent.futures
import time

start = time.perf_counter()

# Function executed by each thread
def work(seconds):
    print(f"Sleeping for {seconds} seconds")
    time.sleep(seconds)
    return "Finished sleeping"

# Create thread pool
with concurrent.futures.ThreadPoolExecutor() as pool:

    # Create multiple tasks using list comprehension
    jobs = [pool.submit(work, 1.5) for i in range(2)]

    # Get completed results
    for job in concurrent.futures.as_completed(jobs):
        print(job.result())

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

Sleeping for 1.5 seconds
 Sleeping for 1.5 seconds
 Finished sleeping
 Finished sleeping
 Finished in 1.51 second(s)

```

In [7]: # Task 6
        # Passing a list of arguments using map

import concurrent.futures
import time

start = time.perf_counter()

# Function that accepts sleeping time
def work(seconds):

```

```

    print(f"Sleeping for {seconds} seconds")
    time.sleep(seconds)
    print("Finished sleeping")

# List of arguments
time_values = [5, 4, 3, 2, 1]

# Create thread pool
with concurrent.futures.ThreadPoolExecutor() as pool:

    # Execute function for each value
    results = pool.map(work, time_values)

    # Display results
    for result in results:
        print(result)

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

```

Sleeping for 5 seconds
Sleeping for 4 seconds
Sleeping for 3 seconds
Sleeping for 2 seconds
Sleeping for 1 seconds
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
None
None
None
None
None
Finished in 5.02 second(s)

```

In [8]: *# Task 7*
Downloading multiple audio files without using threads

```

import requests
import time

# List of media URLs
files = [
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/iphone-remix-6802",
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/flute-62874.mp3",
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/god-flute-2922.mp",
    "https://codeskulptor-demos.commondatastorage.googleapis.com/pang/paza-modul",
    "https://commondatastorage.googleapis.com/codeskulptor-assets/sounddogs/soun
]

start = time.perf_counter()

# Download each file one by one
for link in files:

    with requests.get(link, stream=True) as response:

```

```
# Check whether the request was successful
response.raise_for_status()

# Get file name from URL
filename = link.split("/")[-1]

if filename == "":
    filename = "downloaded_file"

# Save the downloaded file
with open(filename, "wb") as file:

    # Download file in small chunks
    for data in response.iter_content(chunk_size=8192):

        if data:
            file.write(data)

    print(f"{filename} downloaded successfully")

end = time.perf_counter()

print(f"Finished in {end - start:.2f} seconds")
```

```
iphone-remix-68028.mp3 downloaded successfully
flute-62874.mp3 downloaded successfully
god-flute-2922.mp3 downloaded successfully
paza-moduless.mp3 downloaded successfully
soundtrack.mp3 downloaded successfully
Finished in 5.27 seconds
```

In []:

Advanced Python Programming Lab Assignment 4

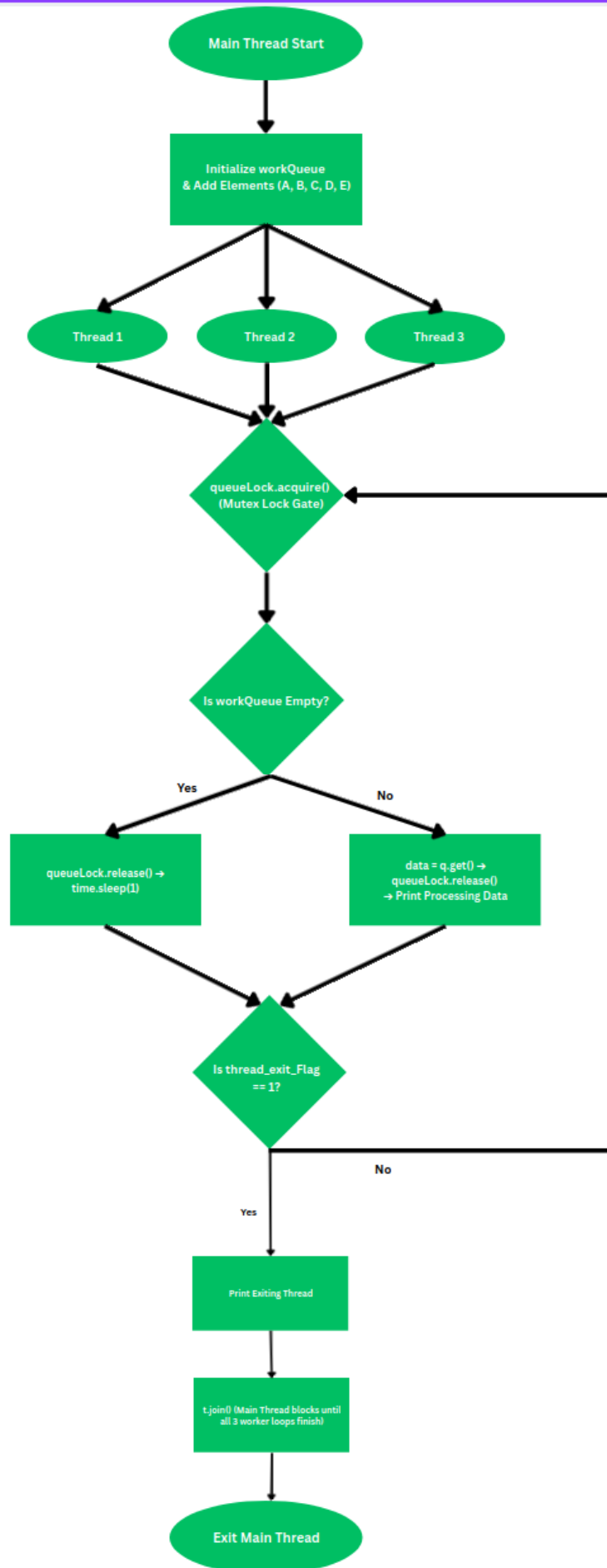
Name: Pranav Anand

Reg No: 23MIC0006

3) Practice printing MainThread name and worker threads

Flowchart in next page

the export quality of canva wasn't good so i had to take 2 ss and align them :(



```
In [9]: #23MIC0006
#Pranav Anand

#Practice printing Main Thread and Worker threads

In [10]: import threading
import time

#1. Define the Worker Function
def worker_task(worker_name):

    current_thread = threading.current_thread()

    print(f"[START] {worker_name} is beginning its task.")
    print(f"Thread Name: {current_thread.name} | Thread ID: {current_thread.ident}\n")

    time.sleep(2)

    print(f"[SUCCESS] {worker_name} has finished execution.\n")

In [11]: #2. Inspect the Main Thread\
main_thread = threading.current_thread()

print("MAIN THREAD INFO")
print(f"Name: {main_thread.name}")
print(f"ID: {main_thread.ident}")
print(f"Is Alive: {main_thread.is_alive()}")
print("\n")

MAIN THREAD INFO
Name: MainThread
ID: 127399511024832
Is Alive: True

In [12]: # 3. Spawn and Start Worker Threads
thread_one = threading.Thread(target=worker_task, args=("Worker-A",), name="CustomThread-1")
thread_two = threading.Thread(target=worker_task, args=("Worker-B",), name="CustomThread-2")

print("Launching Workers")
\
thread_one.start()
thread_two.start()

Launching Workers
[START] Worker-A is beginning its task.
Thread Name: CustomThread-1 | Thread ID: 127399153870400

[START] Worker-B is beginning its task.
Thread Name: CustomThread-2 | Thread ID: 127399145477696

[SUCCESS] Worker-A has finished execution.

[SUCCESS] Worker-B has finished execution.

In [13]: # 4. Synchronize Threads (Join)
thread_one.join()
thread_two.join()

print("ALL THREADS COMPLETE")
print("The Main Thread is now exiting safely.")

ALL THREADS COMPLETE
The Main Thread is now exiting safely.
```